

Chapter 18: Generics

By Ellen Shapiro

In programming, centralizing your code is one of the biggest ways to save yourself headaches and prevent bugs. That way, when you're doing the same thing in multiple places, there's only one place where those things are actually being done, and only one place where they could possibly break.

A really helpful feature of Kotlin for this is called **generics**. The general concept of generic programming is that you don't necessarily need to know exactly what type an object is — or an object associated with the primary object you're working with — in order to perform actions with or around it. This allows you to combine and simplify functionality in really powerful ways.

Anatomy of standard library generic types

When getting started with generics, it helps to look at the major generic types that are included in Kotlin's standard library. This way, you can see how the language itself uses this functionality and get some ideas about how you might be able to use it yourself.

Lists

You've probably noticed working with **List** objects that you sometimes need to declare them with the type of item you expect in the list in angle brackets, such as `List<String>`, or `List<Int>`.

The primary declaration of `List` looks like this, as of Kotlin 1.2:

```
interface List<out E> : Collection<E>
```

Take this apart a bit:

- This is a declaration of an interface, where anything conforming to this interface must be a `Collection`.
- Both `Collection` and `List` have an `E` in angle brackets — this is called the **generic type**. Since it's the same in both places, this indicates that the underlying type of the list and the collection must be the same. You'll delve a bit more into passing generics through to interfaces later in the chapter.
- You'll get to what the `out` bit means much later in the chapter but, for now, you can ignore it.

A lot of the time, you'll see `T` rather than `E` used as the single letter to represent a single generic type. You'll even sometimes see something more elaborate like `Element`. The letter or word representing the generic type is meant more as a hint of what that the type should be, rather than an explicit declaration.

As long as the name of a generic type doesn't collide with the names of any of your classes, you can name a generic type whatever you want. You can think of `T` or `E` or `Element` as a blank, waiting to be filled in with a real, concrete type.

Note: The angle brackets indicating a type is generic were brought over most directly from Java, but this same style is also used in many other programming languages, such as Apple's Swift language.

Now, it's time to play around a bit with Lists in code. In **main.kt**, replace the contents of the `main()` function with:

```
val names: List<String> = listOf("Bob", "Carol", "Ted", "Alice")
println("Names: $names")
val firstName = names.first()
```

So what is `first()` doing under the hood? Take a look at its function declaration:

```
fun <T> List<T>.first(): T
```

There are three uses of `T` as a generic type here:

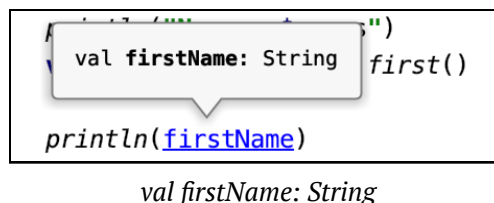
1. The first `<T>` indicates that this is going to be a function that does something with generic type `T`.
2. The second `<T>` indicates that the `List` you're calling this function on must be a list containing only objects of that generic type.
3. The third `T` indicates that the return value will be of the same type `T`, which is contained in the list. It doesn't have angle brackets because it's just the simple underlying type of `T` being returned.

The under-the-hood implementation of this function doesn't need to know or care what type `T` is; it just needs to know it's the **same** type in all three places.

You can see this at work by starting to add the following line to **main.kt**:

```
println(firstName)
```

As you type `firstName`, you'll notice that the compiler has already inferred what type it is, without you explicitly declaring the type of `firstName`:



The screenshot shows a code editor with the following code:

```
val names: List<String> = listOf("Bob", "Carol", "Ted", "Alice")
println("Names: $names")
val firstName = names.first()
println(firstName)
```

A tooltip is displayed over the `firstName` variable in the `println(firstName)` line. The tooltip contains the text:

```
val firstName: String
```

Below the tooltip, the text `val firstName: String` is shown, indicating the inferred type of the variable.

Next, run the program by pressing the Play button in the left sidebar next to the start of the `main()` function, and you'll see the following print out:

```
Names: [Bob, Carol, Ted, Alice]
Bob
```

The first line prints out the entire list and the second line is the first name in the list you created, which you accessed using `first()` — neat! Even neater is the way that type inference can still work even with generics. Delete the `: List<String>` from the first line so that it reads:

```
val names = listOf("Bob", "Carol", "Ted", "Alice")
```

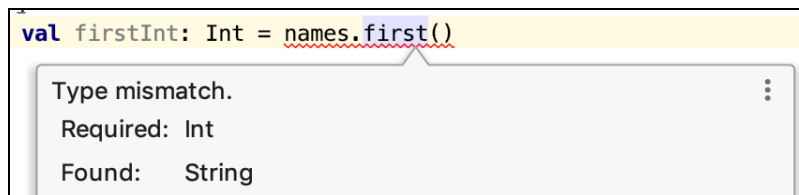
Run the program again, and you'll see the exact same output!

Under the hood, the compiler has realized that, since the items you passed into `listOf()` were all strings, it's creating a `List<String>` without you having to do anything else.

Even without the explicit declaration of the generic type, you can still use its protections. Add the following line to the bottom of `main.kt`:

```
val firstInt: Int = names.first()
```

Try to run again, and you'll see an error:



Type mismatch. Required: Int, Found: String

Even though you haven't explicitly told the compiler that `names` contains nothing but `String` objects, it's still used type inference to say hey, you can't get an `Int` out of a list that only contains `String` objects.

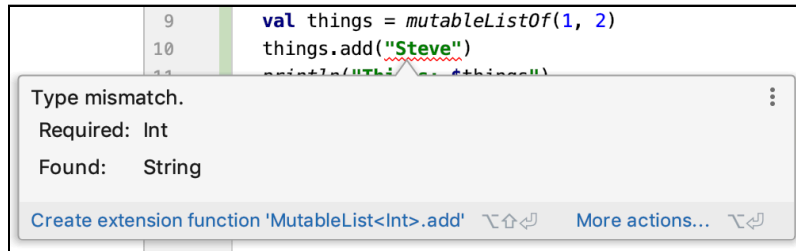
Note: Make sure to comment out or delete the `firstInt` line before proceeding to the next step so that the app continues to compile properly.

Now, as you might remember from Chapter 8, “Arrays & Lists,” a `List` is read-only. In order to add stuff to the list, it must be a `MutableList`.

In `main.kt`, add the following lines to the bottom of the `main()` function:

```
val things = mutableListOf(1, 2)
things.add("Steve")
println("Things: $things")
```

As you might expect due to type inference, you'll see an error when you try to add "Steve":



Type mismatch. Required: Int, Found: String

Since Kotlin's type inference system only sees the two `Int` objects passed into `mutableListOf`, it assumes the generic type for the `MutableList` being created is `<Int>`.

This is great if you want to prevent someone from accidentally adding an object of the wrong type to your list. But what if you **want** to be able to add an object of any type to that mutable list?

Fortunately, there's a type for that: `Any`. This is the superclass of every single class in Kotlin, which means that anything can be stuck into an array whose generic type is `Any`.

To do this, you'll have to tell the compiler explicitly that you want to use this type. You can do that one of two ways. You can either specify the type of the variable at the point where it's declared like this:

```
val things: MutableList<Any> = mutableListOf(1, 2)
```

Alternately, you can save yourself a couple of keystrokes and the explicit type declaration, and pass `Any` into the generic type on `mutableListOf` like so:

```
val things = mutableListOf<Any>(1, 2)
```

Either of these will work but, for now, replace the declaration of `things` in `main.kt` with the `mutableListOf<Any>` version, so that the list knows it is of `Any` type. Now, the program should compile again. Build and run, and you'll see at the bottom of the printout:

```
Things: [1, 2, Steve]
```

We'll come back to playing around with lists in a bit, but first take a look at another major use of generic types in the standard library: Maps.

Maps

Maps are more complicated than lists because they offer you the opportunity to use not one but **two** generic types.

As you saw in Chapter 9, “Maps & Sets,” a map is an object that contains **keys** and **values**. So you probably won’t be too surprised to see the declaration for its interface:

```
interface Map<K, out V>
```

Again, we’ll come back to the out notation later in the chapter, but you’ve probably guessed that K is the generic type of the keys, and V is the generic type of the values.

This allows you to do some fun things like assigning all keys as a specific type, but that values can be of any type.

For instance, at the bottom of **main.kt**’s `main()` function, create a map with several pair objects, the first item of which is always a string:

```
val map = mapOf(  
    Pair("one", 1),  
    Pair("two", "II"),  
    Pair("three", 3.0f)  
)
```

Type inference allows the compiler to figure out that this is a `Map<String, Any>`. Once it’s done that, you’ll get more type safety benefits. For instance, if you try to access something with string keys using an integer, it won’t work. Add the following to the bottom of the `main()` function:

```
val one = map.get(1)
```

You’ll immediately see this warning:

```
val one = map.get(1)  
Type inference failed. The value of the type parameter K should be mentioned in input types  
(argument types, receiver type or expected type). Try to specify it explicitly.
```

Type inference failed. The value of the type parameter K should be mentioned in input types. Try to specify it explicitly.

This also applies when using subscripting to try to access the values of the array. Replace the previous line with:

```
val one = map[1]
```

```
val one = map[1]
```

Type inference failed. The value of the type parameter K should be mentioned in input types (argument types, receiver type or expected type). Try to specify it explicitly.

Type inference failed. The value of the type parameter K should be mentioned in input types. Try to specify it explicitly.

This is extra helpful with subscripting, in the event that you forget whether the value you're attempting to subscript is a `Map` or a `List`. If you use the wrong type (and the type of `K` is not `Int`), the compiler will let you know right away!

Note: Comment out or delete the line with the warning before you proceed.

Another nice feature of maps you can take advantage of with generics is that you can do things based on the type of the keys or the values, since each can be accessed separately.

Since all keys must be unique, you can access them as a `Set<K>`. Since values don't need to be unique, they are returned as a `Collection<V>`.

In the case of this `Map`, keys will be a `Set<String>` and values will be a `Collection<Any>`. You can then do interesting things based on the fact that you know everything in `map.keys` is a `String`.

Add the following lines to the bottom of the `main()` function:

```
val valuesForKeysWithE = map.keys
    .filter { it.contains("e") }
    .map { "Value for $it: ${map[it]}" }
println("Values for keys with E: $valuesForKeysWithE")
```

Build and run the `main()` function, and you'll see:

```
Values for keys with E: [Value for one: 1, Value for three: 3.0]
```

Now, you're printing only the items in `Map`, which have a letter `e` in their key.

While types other than `String` can be used for keys and types other than `Any` can be used for values, the real power of generics lies in what you can do with them when you start to use them in your own types and functions. A good place to start with that is adding an extension function on something that already has a generic constraint.

Extension functions on types with generic constraints

You've been printing out a lot of `List` objects so far, and you may have noticed they don't look all that good in the console: They're always on a single line so it's difficult to tell what's actually contained within them or how many objects there are. Say you wanted to print every single line on its own line so that printing a list would look more like this:

```
- First Item
- Second item
- Third Item
```

The best way to write generic functions is to start simple: Write a function for a case you know you definitely have, using the actual types you need.

At the top of **main.kt**, above the `main()` function declaration, add the following lines:

```
fun List<String>.toBulletedList(): String {
    val separator = "\n - "
    return this.map { "$it" }.joinToString(separator, prefix =
separator, postfix = "\n")
}
```

This is an extension function on a `List` of `String` objects, which will add the bullets to the list as indicated above.

Next, update the first `println` statement in `main()` that prints out names to use this new function:

```
println("Names: ${names.toBulletedList()}")
```

Since the `println` statement for `valuesForKeyWithE` is printing a list of `Strings`, that also can be updated:

```
println("Values for keys with E: $
{valuesForKeyWithE.toBulletedList()}")
```

Build and run again, and you'll see the updated printouts:

```
Names:
- Bob
- Carol
```



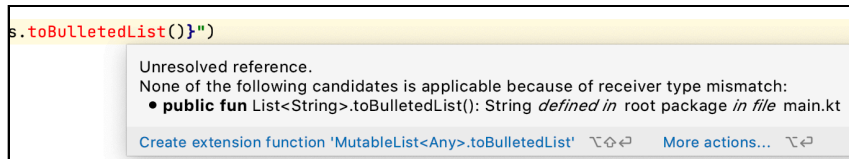
```
- Ted
- Alice
```

```
Bob
Things: [1, 2, Steve]
Values for keys with E:
- Value for one: 1
- Value for three: 3.0
```

It's working for the two lists of `String` objects, but what about the things list, which has objects of type `Any`? Go to where that line is printed, and update it to:

```
println("Things: ${things.toBulletedList()}")
```

Immediately, you'll see an error:

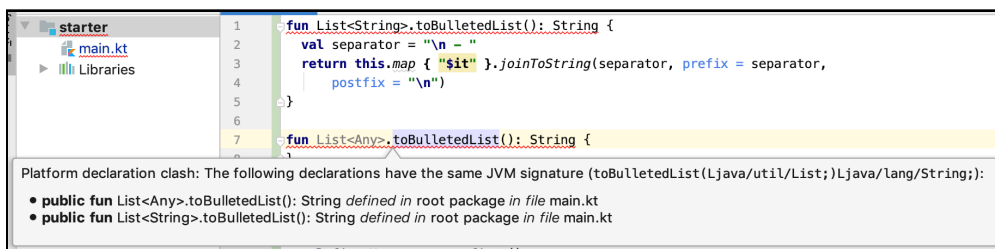


Unresolved reference.

Since `things` is of type `Any`, rather than `String`, it can't use the existing extension method. Time to make it more generic! Start by adding the same functionality for lists with items of type `Any`! Underneath the initial extension function, start trying to add a new function:

```
fun List<Any>.toBulletedList(): String {
}
```

However, before you even add the function body, there's a major error happening:



Platform declaration clash: The following declarations have the same JVM signature

The two declarations have the same signature, even though they are using different types for the placeholder type — so the Kotlin compiler can't easily tell which one you're going to use.

Fortunately, since there's nothing in `toBulletedList()` that actually requires anything to be a `String`, you can quickly turn it into a generic function that can be used on a list of whatever type you want!

First, delete or comment out the `Any` function you just added. Next, update the type of the `List` on `toBulletedList()` from `String` to a generic `T`:

```
fun List<T>.toBulletedList(): String
```

Now, at this point, the compiler freaks out a bit because it has no idea what `T` is:



Unresolved reference: T

Fortunately, this is easy to solve: You just need to let the compiler know that this is a generic parameter by adding `<T>` directly after the `fun` keyword:

```
fun <T> List<T>.toBulletedList(): String
```

Once you do that, all of your errors should be resolved. Build and run the program again. This time where things is printed out, you'll also see a nice, pretty bulleted list:

```
Things:
- 1
- 2
- Steve
```

Creating your own generic constraints

Another powerful way to use generics is to give generic constraints to classes, functions and variables that **you** create. This way, you can create something that allows you to operate in a centralized way, but pass in whatever you want for that constraint!

A good place to think about a class with a generic constraint is someplace wherein the fundamental operations happening don't necessarily need to know what kinds of things are happening under the hood.

Moving between an old house or apartment and a new one is a good example. This can be really expensive. Even if you're moving within the same city, you generally have a lot of stuff that you need to move out of your old place and into a truck, and then from that truck into your new place.

However, different moving companies have different specialties, and hiring one that gives you way more protection than you need (or not enough protection) for a particular item can make your move much more expensive than it needs to be.

Fundamentally, however, the same thing happens with everything you move in a situation like this: Things from your old place are moved out and into a truck, the truck goes to your new place and then those things are moved out of the truck and into the new place.

Generally, when you can boil down the description of what's affected under the hood to "things," you've got a good chance that generics could be useful.

Below the `toBulletedList()` extension function and above the `main()` function, add a new generic `Mover` class, which allows you to move a passed-in type of item:

```
// 1
class Mover<T>{
    // 2
    thingsToMove: List<T>,
    val truckHeightInInches: Int = (12 * 12)
} {

    // 3
    private var thingsLeftInOldPlace = mutableListOf<T>()
    private var thingsInTruck = mutableListOf<T>()
    private var thingsInNewPlace = mutableListOf<T>()

    // 4
    init {
        thingsLeftInOldPlace.addAll(thingsToMove)
    }

    // 5
    fun moveEverythingToTruck() {
        while (thingsLeftInOldPlace.count() > 0) {
            val item = thingsLeftInOldPlace.removeAt(0)
            thingsInTruck.add(item)
            println("Moved your $item to the truck!")
        }
    }

    // 6
    fun moveEverythingIntoNewPlace() {
        while (thingsInTruck.count() > 0) {
```

```

        val item = thingsInTruck.removeAt(0)
        thingsInNewPlace.add(item)
        println("Moved your $item into your new place!")
    }
}

// 7
fun finishMove() {
    println("OK, we finished! We were able to move your:$
{thingsInNewPlace.toBulletedList()}")
}
}

```

What's happening in this code?

1. By giving the Mover class a generic constraint of <T>, you're saying that anything that creates an instance of this class must fill in the blank for what type T actually is.
2. The constructor receives a List of the same T type that your mover class's generic constraint is, along with the height of the truck in inches, with a default value of 12-feet tall.
3. Some MutableLists are declared in order to handle what items are where: in your old place, in your new place or in the Mover's truck.
4. The init function takes the passed-in list of items to move from the constructor and adds all of them to the list of things in the old place.
5. A function is added with a loop to move all items from the old place into the truck.
6. A function is added with a loop to move all items from the truck into the new place.
7. A list of what was moved is printed out using finishMove().

One thing you'll notice that is **not** in the Mover<T> class: any kind of information about what underlying type T could possibly be.

In the moving analogy, if you have some stuff that's big but cheap, you can usually wind up hiring some cheaper movers to move them. You'll do that next.

To start, define a simple class below Mover<T> to represent a cheap thing you want moved:

```

class CheapThing(val name: String) {
    override fun toString(): String {

```

```
        return name
    }
}
```

This class doesn't do much besides hang on to the name of the item you're moving and use that name instead of the object's address in memory when the object is printed.

Next, go to the `main()` function and add the following lines at the bottom of the file:

```
val cheapThings = listOf(
    CheapThing("Cinder Block table"),
    CheapThing("Box of old books"),
    CheapThing("Ugly old couch")
)
val cheapMover = Mover(cheapThings)
```

These lines create a list of things and use that list to create a `Mover` object. Note that, because of type inference with your list is of type `List<CheapThing>`, Kotlin knows that your mover is of type `Mover<CheapThing>`.

Underneath that declaration, call the three functions that will actually move all your stuff, and complete the move:

```
cheapMover.moveEverythingToTruck()
cheapMover.moveEverythingIntoNewPlace()
cheapMover.finishMove()
```

Build and run the program. In the console, you should see:

```
Moved your Cinder Block table to the truck!
Moved your Box of old books to the truck!
Moved your Ugly old couch to the truck!
Moved your Cinder Block table into your new place!
Moved your Box of old books into your new place!
Moved your Ugly old couch into your new place!
OK, we finished! We were able to move your:
- Cinder Block table
- Box of old books
- Ugly old couch
```

Without the `Mover` class knowing anything about what type of object is being moved, you were able to create a `Mover` object and have it move all your cheap things!

Unlike big cheap objects, you'll almost always want to hire professional movers to move your breakable things. These sorts of movers might be expensive, but they'll usually keep things from breaking (or they will replace them if they do break).

Below your CheapThing class, define a simple class to represent something that's breakable, along with a way to "break" it:

```
class BreakableThing(  
    val name: String,  
    var isBroken: Boolean = false  
) {  
    fun smash() {  
        isBroken = true  
    }  
  
    override fun toString(): String {  
        return name  
    }  
}
```

Next, back at the bottom of the main() function, add some breakable things and an expensive mover to move them:

```
val television = BreakableThing("Flat-Screen Television")  
val breakableThings = listOf(  
    television,  
    BreakableThing("Mirror"),  
    BreakableThing("Guitar")  
)  
val expensiveMover = Mover(breakableThings)
```

Then, call the same functions you called on cheapMover to tell the expensive mover to move your breakable things:

```
expensiveMover.moveEverythingToTruck()  
expensiveMover.moveEverythingIntoNewPlace()  
expensiveMover.finishMove()
```

Build and run again, and the following will print on the console:

```
Moved your Flat-Screen Television to the truck!  
Moved your Mirror to the truck!  
Moved your Guitar to the truck!  
Moved your Flat-Screen Television into your new place!  
Moved your Mirror into your new place!  
Moved your Guitar into your new place!  
OK, we finished! We were able to move your:  
- Flat-Screen Television  
- Mirror  
- Guitar
```

Well, that looks about the same as the output for moving cheap things! But what happens when something breaks?

Between the line moving everything into the truck and the line moving everything into the new place, add the following line:

```
television.smash()
```

Build and run... and it prints out exactly the same lines as above. Uh oh— that expensive mover isn't actually doing anything to find out if something is broken!

That's because there's nothing in the `Mover` class that allows the mover to check if something is broken. How can we make the `Mover` class do that? One way to do it is with smart casts.

Update `moveEverythingToTruck()` to read as follows:

```
fun moveEverythingToTruck() {
    while (thingsLeftInOldPlace.count() > 0) {
        val item = thingsLeftInOldPlace.removeAt(0)

        if (item is BreakableThing) {
            if (!item.isBroken) {
                thingsInTruck.add(item)
                println("Moved your $item to the truck!")
            } else {
                println("Could not move your $item to the truck")
            }
        } else {
            thingsInTruck.add(item)
            println("Moved your $item to the truck!")
        }
    }
}
```

This works! The `(item is BreakableThing)` check makes everything within that if expression aware that the item is of that specific type.

But there are a couple of things that are highly problematic from both a conceptual and practical standpoint about this code:

- A class with a generic constraint shouldn't need to know what specific type `T` is in order to be able to do things with it — but, here, it does. If it has to know what subtype it's holding, the point of generics is somewhat defeated.
- A bunch of logic gets exactly repeated — often a sign that copy-pasting of code was employed. This is dangerous, because it means that whatever bugs were in the code which was copy-pasted also got copy-pasted!

So how can we further constrain the `<T>` generic type on the `Mover` class so that we know that it can always be checked, without having to make everything descend from the same superclass?

The answer: Interfaces!

Interfaces

Interfaces allow you to declare information about what something **does**, rather than what it **is**, as a class hierarchy would.

Above the declaration of the `Mover<T>` class, add a new interface.

```
interface Checkable {  
    fun checkIsOK(): Boolean  
}
```

Next, update the generic constraint (i.e., the bit in the angle brackets) of the `Mover` class so it only accepts types that conform to the `Checkable` interface:

```
class Mover<T: Checkable>
```

This updated constraint means that attempting to create a `Mover` with a class that does not conform to `Checkable` will fail at compile time. Before continuing, add one more private `var` to the `Mover` class below the other three to hold things that fail the check:

```
private var thingsWhichFailedCheck = mutableListOf<T>()
```

Next, update the `moveEverythingToTruck()` function to take advantage of the `Mover` class's new knowledge that anything it's receiving has to be of the type `Checkable`:

```
fun moveEverythingToTruck() {  
    while (thingsLeftInOldPlace.count() > 0) {  
        val item = thingsLeftInOldPlace.removeAt(0)  
  
        if (item.checkIsOK()) {  
            thingsInTruck.add(item)  
            println("Moved your $item to the truck!")  
        } else {  
            thingsWhichFailedCheck.add(item)  
            println("Could not move your $item to the truck :[")  
        }  
    }  
}
```

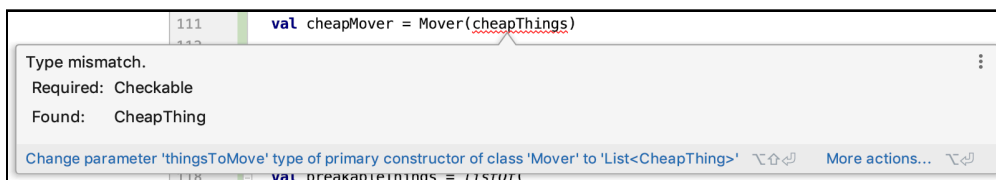

Next, update `moveEverythingIntoNewPlace` to also take advantage of this new ability to check if something is okay:

```
fun moveEverythingIntoNewPlace() {
    while (thingsInTruck.count() > 0) {
        val item = thingsInTruck.removeAt(0)
        if (item.checkIsOK()) {
            thingsInNewPlace.add(item)
            println("Moved your $item into your new place!")
        } else {
            thingsWhichFailedCheck.add(item)
            println("Could not move your $item into your new place :")
        }
    }
}
```

Next, update the `finishMove` function so that your `Mover` lets their customer know what items were not moved successfully, if there were any that weren't moved:

```
fun finishMove() {
    println("OK, we finished! We were able to move your:$")
    {thingsInNewPlace.toBulletedList()})
    if (thingsWhichFailedCheck.isNotEmpty()) {
        println("But we need to talk about your:$")
        {thingsWhichFailedCheck.toBulletedList()})
    }
}
```

You've now updated the `Mover` class to handle only this type. You'll still see two errors in your `main()` function, both of which look something like this:



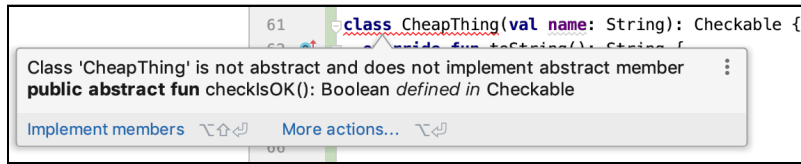
Type mismatch. Required: Checkable, Found: CheapThing

This is because now that `Mover` only accepts types conforming to `Checkable`, it can't accept either `CheapThing` or `BreakableThing` if they don't conform to `Checkable`.

To fix this, first, you need to update `CheapThing` to conform to `Checkable`. Update the declaration:

```
class CheapThing(val name: String): Checkable
```

This will immediately throw up an error that you need to add the function that Checkable declares will be there if something conforms:



Class 'CheapThing' is not abstract and does not implement abstract member checkIsOK

To fix this error, at the bottom of the CheapThing class, add the following line:

```
override fun checkIsOK(): Boolean = true
```

Here, you're overriding the checkIsOK() function defined in the Checkable interface. However, since you don't really want your movers worrying about whether a cheap thing is OK, your implementation says, "You know what? It's always OK."

Next, go to your BreakableThing class and also add conformance to Checkable in the declaration:

```
class BreakableThing(
    val name: String,
    var isBroken: Boolean = false
): Checkable
```

Now, at the bottom of the BreakableThing class, add an override of the checkIsOK() function which actually does a bit of checking to make sure something is OK:

```
override fun checkIsOK(): Boolean {
    return !isBroken
}
```

Build and run, and while you'll still see the same output from the cheapMover, the expensiveMover will now actually be performing the check you want — and show that your TV got smashed after it got put onto the truck:

```
Moved your Flat-Screen Television to the truck!
Moved your Mirror to the truck!
Moved your Guitar to the truck!
Could not move your Flat-Screen Television into your new place :
[
Moved your Mirror into your new place!
Moved your Guitar into your new place!
OK, we finished! We were able to move your:
- Mirror
```

– Guitar

But we need to talk about your:

– Flat-Screen Television

You’ve used an interface to give your generic types more power. But can you go one level deeper: What about making a generic interface?

Generic interfaces

A generic interface is an interface that is constrained to a generic type. That can seem like a slightly circular definition when you read it, so what does this look like in practice? Keep going with the moving metaphor.

Often, when you’re moving, you will put one or more things into a box or a plastic tub or some other sort of container to make it easier or safer to move your stuff.

If you want to say that a particular type of thing can only be moved in a particular type of container, you can easily represent this with a generic interface.

Above the `main()` function, add a new interface for a typed container:

```
// 1
interface Container<T> {
// 2
    fun canAddAnotherItem(): Boolean
    fun addItem(item: T)
// 3
    fun canRemoveAnotherItem(): Boolean
    fun removeItem(): T
// 4
    fun getAnother(): Container<T>
// 5
    fun contents(): List<T>
}
```

You’ve created a generic interface with several methods — some of which accept generic values and some of which return generic values. What’s happening here?

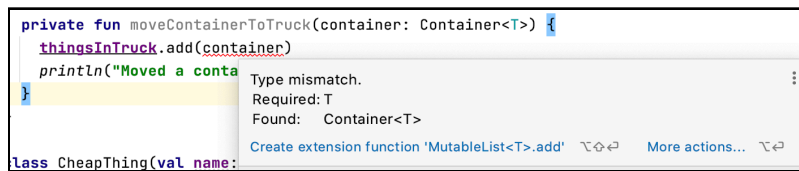
1. You’ve declared that your interface needs a generic type passed into it whenever a class implementing this interface is created.
2. You’ve created functions to check whether another item can be added to the container, and then pass an item of the generic type of the container in to be added.

3. You've added functions to do the opposite: checking if there are any more items to remove from the container and then to return an item of the generic type as it's removed from the container.
4. You've added a generic factory method to get a new, empty container. This will help if your container fails the `canAddAnotherItem()` check.
5. You've added a way access a typed list of what items are in the container.

In the `Mover` class, add a new method below `moveEverythingIntoTruck` to move a generic container into the truck:

```
private fun moveContainerToTruck(container: Container<T>) {
    thingsInTruck.add(container)
    println("Moved a container with your $
    {container.contents().toBulletedList()} to the truck!")
}
```

You'll see an error show up on the line where you're trying to move the container to the truck:



Type mismatch. Required: T, Found: Container<T>

While a generic container takes the same `<T>` that is being used for the mover class as its type, it's not actually of that type, so trying to add it to the truck will fail.

To fix this, update the type of `thingsInTruck` so that it can accept an object of Any type:

```
private var thingsInTruck = mutableListOf<Any>()
```

This will cause a few errors in `moveEverythingIntoNewPlace()`, which we'll return to shortly. What this does do is resolve the error in adding a container to the truck, allowing you to proceed with updating `moveEverythingToTruck()` so that it can add items to a container if one is provided.

First, update the method signature to take a nullable container, typed to the same generic type as is being passed in to the `Mover<T>` class:

```
fun moveEverythingToTruck(startingContainer: Container<T>?)
```

Next, within the method, at the very top above the while loop, add a variable to hang on to whatever the current container is, if it exists:

```
var currentContainer = startingContainer
```

And at the very bottom below the while loop, add a line moving the current container to the truck, if it exists:

```
currentContainer?.let { moveContainerToTruck(it) }
```

Within the `if (item.checkIsOK()) {...}` block where you were previously moving an item to the truck directly, add the following code, so that if a container is provided, you provide logic to pack the item into a container (moving a full container and getting a new one if necessary):

```
// 1
if (currentContainer != null) {
    // 2
    if (!currentContainer.canAddAnotherItem()) {
        moveContainerToTruck(currentContainer)
        currentContainer = currentContainer.getAnother()
    }
    // 3
    currentContainer.addItem(item)
    println("Packed your $item!")
} else {
    // 4
    thingsInTruck.add(item)
    println("Moved your $item to the truck!")
}
```

What's happening in this code?

1. You're checking whether the current container is null. If it isn't, then everything in the `if` block will be smart cast so that `currentContainer` can be accessed without a null check. If it is null, you go to the `else` block (#4).
2. You're checking if the current container is full. If it's full, you move it to the truck and get another container. If it's not, you just keep going.
3. You're adding the item to the `currentContainer`, which may or may not have been replaced.
4. If you're in this block, `currentContainer` was null and you just keep putting items directly in the truck as you were before.

Now, it's time to move on to getting everything back out of the truck and/or whatever container it was packed in. And this is where things start to get pretty complicated because of something Kotlin brought over from Java: type erasure.

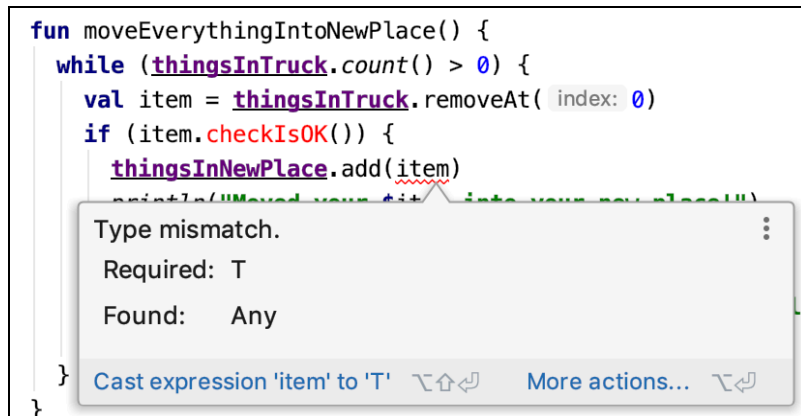
Type erasure

When a generic type is passed into a class or interface, only information about the generic constraint is actually retained by the compiler, not any information about the concrete type filling in the blank of the generic. This is known as **type erasure**.

When your generic constraint has an interface that it must adhere to, any functions defined in the interface are available in the interface or class using the generic type. But what if you want to do more than that?

In this case, you've updated the generic constraint of `thingsInTruck` to be `<Any>`, but the things that can go in the apartment still must be of the type being passed in to `Mover<T>`.

Go to `moveEverythingIntoNewPlace()`. You'll see the errors that you ignored earlier:



Type mismatch: Required: T, Founds: Any

First, clear up these errors by extracting the bit checking the item into its own private function, passing the item as a generically typed parameter:

```
private fun tryToMoveItemIntoNewPlace(item: T) {
    if (item.checkIsOK()) {
        thingsInNewPlace.add(item)
        println("Moved your $item into your new place!")
    }
}
```

```

    } else {
        thingsWhichFailedCheck.add(item)
        println("Could not move your $item into your new place :[")
    }
}

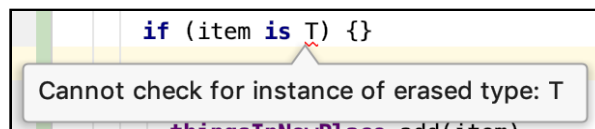
```

Now that the compiler has some assurance that something passed into that function is of the correct type, those errors will go away. So, from there, how do you actually figure out how to get items either out of containers or directly out of the truck, so that they can be passed into that function?

Back in `moveEverythingToNewPlace()`, right below where you remove the item from the truck, try to determine whether the item is of the correct generic type by checking whether it's an instance of that type:

```
if (item is T) {}
```

As the compiler will tell you, type erasure means that this is not possible:



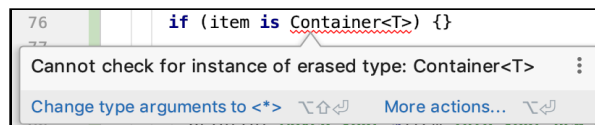
Cannot check for instance of erased type: T

Because the compiler doesn't actually know what type `T` is, you can't use `is` to check its type.

Instead, try the opposite approach: Instead of checking what things in the truck are type `T`, see which ones are of type `Container<T>` by replacing the erroring line with:

```
if (item is Container<T>) {}
```

This will give you another error about erased types:



Cannot check for instance of erased type: Container<T>

This error might seem weird, because the compiler definitely knows what a `Container` is — it just doesn't know what type `T` is. This is where a new tool comes in handy: star projection.

Star projection

Replace the T in Container<T> with an asterisk:

```
if (item is Container<*>) {}
```

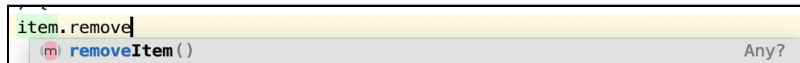
Now, the error will go away. This is some dark magic known as **star projection**. The “star” part is named after the asterisk.

The “projection” part means that while you know that Container accepts a generic type, you have no idea what that generic type will be. This is a way to tell the compiler, “This could be a Container of anything, but I can tell you that it’s definitely a Container.”

Now that you can actually access the container, try to remove an item from it:

```
if (item is Container<*>) {  
    val itemInContainer = item.removeItem()  
}
```

You’ll notice a couple things, here. First, smart casting gives you access to the functions defined in Container. Second, when you try to use the function, the type isn’t quite what you want it to be:



```
item.remove  
(m) removeItem() Any?
```

Instead of T, it’s a nullable Any?. This is the essence of star projection’s workaround for type erasure: Instead of actually getting information about the generic type, it assumes it must be... some type, which could be nullable.

So this isn’t ideal for what you want to do here either, because you still don’t know what type is in the container you’ve been able to extract.

What you really need is even darker magic: **reified** type parameters.

Reified type parameters

Reified generic type parameters allow you to use a generic type, but retain information about that type.

You’re going to use a function from the Kotlin standard library, which uses a reified generic type. This is the declaration of that function:

```
inline fun <reified R> Iterable<*>.filterIsInstance(): List<R>
```


There are several things going on in this declaration:

- The `inline` declaration tells the compiler that any calls to this method must be compiled inline, so that it's still possible to access the type information of the generic type.
- The `reified` declaration tells the compiler, ok, *actually* hang on to the type information of the generic type being passed in here. This can cause a performance hit, so you must explicitly opt-in to make this happen.
- The `Iterable<*>` in this function is being called on is a star-projected `Iterable` type. Essentially, anything which can be iterated through, with any kind of generic type, can theoretically use this function (you'll see an exception to this shortly).
- Finally, the return type is a `List` of items in the `Iterable`, which are of the reified generic type parameter that was passed in.

What does this look like in code? Go back to `moveEverythingIntoNewPlace()` and, at the very top of the method, add a new line:

```
val breakableThings =
    thingsInTruck.filterIsInstance<BreakableThing>()
```

Below that, start to type `break` to access the instance you just created. You'll see that because of the reified type parameter, the blank of `filterIsInstance`'s `List<R>` return type has been filled in by `BreakableThing`:

```
fun moveEverythingIntoNewPlace() {
    val breakableThings = thingsInTruck.filterIsInstance<BreakableThing>()
    break
    v breakableThings                                List<BreakableThing>
```

That's how a reified type works when you pass in a concrete type. What happens if you try to pass in a generic type? Comment out or delete the lines you've just added and replace them with a line attempting to filter for items of `Mover<T>`'s generic type:

```
val items = thingsInTruck.filterIsInstance<T>()
```

Unfortunately, this still doesn't work because of type erasure:

```
val items = thingsInTruck.filterIsInstance<T>()
// Cannot use 'T' as reified type parameter. Use a class instead.
```

Cannot use 'T' as reified type parameter. Use a class instead.

The type of `T` was already erased by the time the compiler gets to this point in the code, so there's no way for it to get that information back, even if it wants to. The compiler doesn't know what `T` is, so it can't check for instances of `T`.

Comment out or delete the line you just added, and see if you can get it to filter out only star-projected containers:

```
val containers = thingsInTruck.filterIsInstance<Container<*>>()
```

Hey, that works! The problem is that you're back where you were with star projection before: The type of `containers` is `List<Any?>`, and you'd still have to do type casting to ensure that you have the correct type.

Just for laughs, try replacing the `*` with `T` — going from star projection to passing the generic type that has been passed into `Mover` into the `Container` you're trying to access:

```
val containers = thingsInTruck.filterIsInstance<Container<T>>()
```

What the... that works?!

While everything else you've tried hasn't worked due to type erasure, here the compiler has just enough information to understand that it needs to get a container of a particular type — but it doesn't actually matter under the hood what type it is.

Now, you're cooking with gas! Because the return type is `List<Container<T>>`, you can once again have type safety when getting items out of the container.

Right below where you got the `containers` list, add the following code to remove all the items from all your containers:

```
for (container in containers) {
    thingsInTruck.remove(container)
    while (container.canRemoveAnotherItem()) {
        val itemInContainer = container.removeItem()
        println("Unpacked your $itemInContainer!")
        tryToMoveItemIntoNewPlace(itemInContainer)
    }
}
```

Again, you'll notice that when you call `container.removeItem()`, because it's a `Container<T>`, the type of `itemInContainer` becomes `T`, and you can pass it to `tryToMoveItemIntoNewPlace` without issue.

But what happens if your items weren't in containers? Anything remaining in the truck should be an item of type T, so you can make some assumptions about what it should be.

Below the for loop emptying the containers, update the while loop to read as follows:

```
while (thingsInTruck.count() > 0) {  
    val item = thingsInTruck.removeAt(0) as? T  
    if (item != null) {  
        tryToMoveItemIntoNewPlace(item)  
    } else {  
        println("Something in the truck was not of the expected  
generic type: $item")  
    }  
}
```

At this point, there really isn't a great way to avoid using the as unchecked cast operator to ensure type safety with T. However, you can suppress any exception that would come up by using the nullable version of this operator, as?, which will return null instead of throwing an exception if the cast fails.

If the as? cast fails, it's now printed out for diagnostic purposes, but it doesn't crash your app.

Note: If the unchecked cast warning that you get here bothers you because you're actually returning null if the item is not of the proper type, you can add a @SuppressWarnings("UNCHECKED_CAST") annotation to the line above the as? cast.

Now, it's *finally* time to add a class, which implements the type Container<T> for a given type.

You probably wouldn't want to bother with a container for your CheapThing objects, but you probably would want to at least put your BreakableThings into a box.

Below the definition of your Container<T> interface, create a CardboardBox implementation of the Container interface which holds BreakableThings:

```
// 1  
class CardboardBox: Container<BreakableThing> {  
    //2  
    private var items = mutableListOf<BreakableThing>()  
  
    override fun contents(): List<BreakableThing> {  
        // 3
```

```
        return items.toList()
    }

    // 4
    override fun canAddAnotherItem(): Boolean {
        return items.count() < 2
    }

    override fun addItem(item: BreakableThing) {
        // 5
        items.add(item)
    }

    override fun canRemoveAnotherItem(): Boolean {
        // 6
        return items.count() > 0
    }

    override fun removeItem(): BreakableThing {
        // 7
        val lastItem = items.last()
        items.remove(lastItem)
        return lastItem
    }

    override fun getAnother(): Container<BreakableThing> {
        // 8
        return CardboardBox()
    }
}
```

What's happening in this code?

1. First, you're declaring a class called `CardboardBox`, which conforms to `Container` and provides `BreakableThing` as the generic type.
2. You're adding a private mutable list to store the items within the `CardboardBox` so that only the `CardboardBox` itself knows about this mutable list.
3. Since `T` has been replaced with `BreakableThing`, you're returning an immutable copy of your mutable list of `BreakableThings` when asked for a list of the contents of the `CardboardBox`.
4. Here, you're assuming that each `CardboardBox` can only fit two things into it. If it's already got two things in it, another item can't be added.
5. You add the passed-in `BreakableThing` to the private mutable list when the `addItem` function is called.

6. You check if there are any more items to remove from the `CardboardBox` — in this case, validating that there are more items in the underlying `items` array.
7. You remove the last item from the underlying `items` array and return it when asked to remove an item from the `CardboardBox`.
8. When asked to create another `Container<BreakableThing>`, you create another `CardboardBox`, since it already conforms to this generic requirement.

Now that you've created a type that implements the `Container` interface, it's time to make it possible to move it!

In the `main()` function, update the `moveEverythingToTruck` call for the cheap mover to explicitly provide a null container (hey, you wanted cheap!):

```
cheapMover.moveEverythingToTruck(null)
```

Next, update the `moveEverythingToTruck` call for the expensive mover to provide a `CardboardBox`:

```
expensiveMover.moveEverythingToTruck(CardboardBox())
```

Build and run your code, and you'll be able to see in the printed logs that your `cheapMover` still moves everything directly into and out of their truck:

```
Moved your Cinder Block table to the truck!
Moved your Box of old books to the truck!
Moved your Ugly old couch to the truck!
Moved your Cinder Block table into your new place!
Moved your Box of old books into your new place!
Moved your Ugly old couch into your new place!
OK, we finished! We were able to move your:
- Cinder Block table
- Box of old books
- Ugly old couch
```

...while your `expensiveMover` packs all your items into containers before moving them:

```
Packed your Flat-Screen Television!
Packed your Mirror!
Moved a container with your
- Flat-Screen Television
- Mirror
to the truck!
Packed your Guitar!
Moved a container with your
- Guitar
```

```
to the truck!
Unpacked your Mirror!
Moved your Mirror into your new place!
Unpacked your Flat-Screen Television!
Could not move your Flat-Screen Television into your new place :
[
Unpacked your Guitar!
Moved your Guitar into your new place!
OK, we finished! We were able to move your:
- Mirror
- Guitar

But we need to talk about your:
- Flat-Screen Television
```

So to recap, you've now got a ton of generic tools in this one example with movers:

- A Checkable interface.
- A Mover with a `<T: Checkable>` constraint, which can move any item that conforms to Checkable.
- A Container<T> class that can move any items of a specific type.
- The ability to have the mover move your items in a Container<T> that uses the same `<T: Checkable>` that was passed into your Mover.
- Use of a standard library method using reified types.
- A nullable unchecked cast that helps determine if something is of the correct type.

Whew! That is a whole lot of stuff. But there's one more thing to discuss before moving on from generics: variance.

Generic type variance (a.k.a., in and out declarations)

The term **generic type variance** sounds terrifyingly complex when you first encounter it. This concept is nowhere near as complicated as it sounds.

There are two types of variance you can declare with a class or an interface that use a generic type:

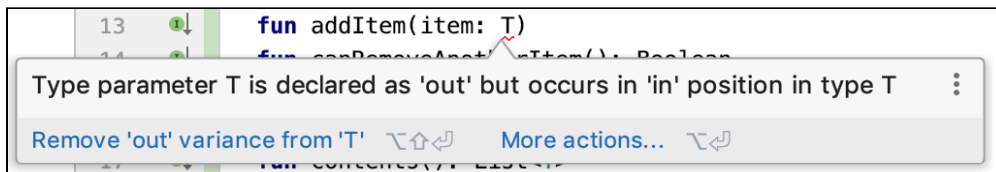
- **in** variance means that the generic type will only ever be used in parameters or other things being handed *into* your type.

- **out** variance means that the generic type will only ever be used in return values or other things coming *out* of your type.

A really quick and easy way to see how this can affect a type you've declared is modifying the **Container** class you declared earlier. Go to the class declaration and add out variance to the generic type by updating the declaration as follows:

```
interface Container<out T>
```

Immediately, the compiler will be unhappy:



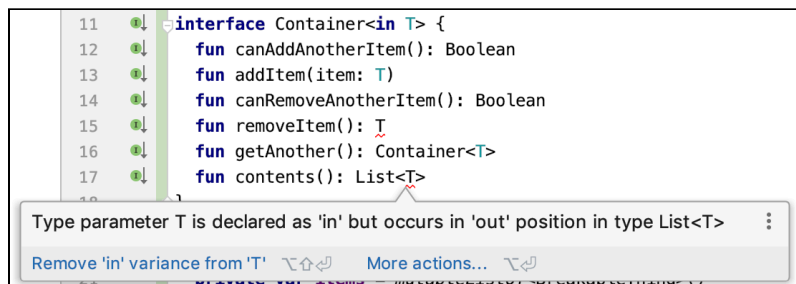
Type parameter T is declared as 'out' but occurs in 'in' position in type T

You've told the compiler that T would only be used in an out position, but this error is telling you that one of the functions you've already declared is being used as something that is passed in.

Now, try doing the opposite: Declaring that T uses in variance. Update the declaration to the following:

```
interface Container<in T>
```

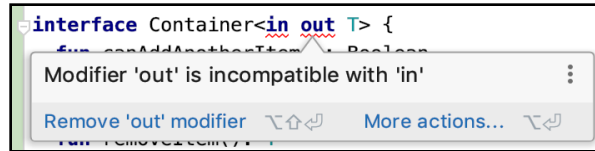
The warning will go away on the function with the generic parameter, but it'll pop right back up in two other places:



Type parameter T is declared as 'in' but occurs in 'out' position in type T

Now, you're seeing that, while the compiler thinks it's OK for a generically typed parameter to be passed into your class or interface, it's not OK to have a return value of that same generic type or to have a return value of another thing referencing that generic type.

You might be tempted to explicitly state that a class or interface has both in and out type variance for its generic type, but, if you give it a try, you'll see that the compiler would prefer that you don't do that:



Modifier 'out' is incompatible with 'in'

If a generic type has both in and out variance, you must leave out both declarations for it to compile. Change Container's interface declaration back to:

```
interface Container<T>
```

Once you do, everything will compile again.

So now that you know **what** this does, there comes a thornier question: **Why** would you want to make this kind of restriction?

You want to make it clear to the caller of your class with a generic constraint, and to the compiler, whether the class is restricted in terms of what it can do with the generic type.

If your type has out variance, you can infer some things automatically, while comparing instances with related types.

For instance, recall that `List<T>` is actually declared as `List<out T>`. Add the following lines to the bottom of the `main()` function:

```
val ints = listOf(1, 2, 3)
val numbers: List<Number> = ints
```

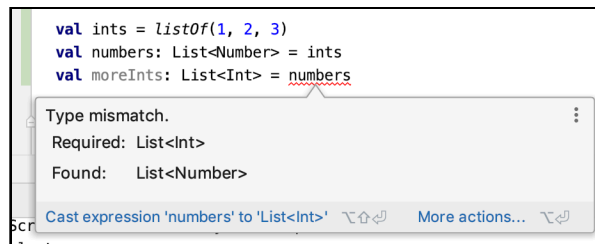
This compiles even though the inferred type of `ints` is `List<Int>`. Since `Int` is a subtype of `Number`, anything that's an `Int` will definitely also be a `Number`.

Since anything being returned from `List<Number>` would also be able to return an `Int`, you can infer that a `List<Int>` can be assigned to a variable with the type `List<Number>` without any problems.

However, this restricts you from doing the opposite. Add a line trying to assign `numbers` to a `List<Int>` variable:

```
val moreInts: List<Int> = numbers
```


This causes a type-checking error:



Type mismatch. Required: List<Int>, Found: List<Number>

If `List<Int>` has something with a return value `Int`, you cannot simply return a `Number` since `Number` is also the supertype of several other types in Kotlin, such as `Float`.

It **could** be an `Int` — but it also could be some other subtype of `Number`. This is why the compiler errors out when you try to assign a `List<Number>` to a variable of type `List<Int>`.

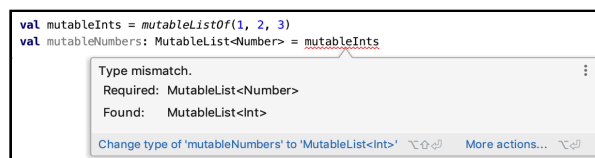
Note: Comment out or delete the line you just added to get rid of the error before continuing.

Contrast `List`'s behavior with that of `MutableList<T>`, which has neither an `in` nor an `out` modifier for its generic type.

Add the following lines to the bottom of your `main()` function:

```
val mutableInts = mutableListOf(1, 2, 3)
val mutableNumbers: MutableList<Number> = mutableInts
```

This errors immediately:



Type mismatch. Required: MutableList<Number>, Found: MutableList<Int>

Because `MutableList` both accepts and returns parameters of type `T`, they always have to be the same type, and you can't make the assumption that you'll be able to use subtypes interchangeably. Therefore `T` must always simply be its own type — not a subtype or a supertype.

There aren't a lot of types that have in variance to give an example with, but one is `Comparator`. Its interface looks like this:

```
interface Comparable<in T> {  
    operator fun compareTo(other: T): Int  
}
```

Since you can't instantiate an interface without a concrete implementation, you're going to create a small function that takes this type as a parameter to examine how this works within your `main()` function.

At the bottom of your `main()` function, add an example function which takes a `Comparable<Number>` and compares it to an `Int` and a `Float`:

```
fun compare(comparator: Comparable<Number>) {  
    val int: Int = 1  
    comparator.compareTo(int)  
    val float: Float = 1.0f  
    comparator.compareTo(float)  
}
```

Something conforming to `Comparable<Number>` can compare itself to both `Int` and `Float` values, since both are subtypes of `Number`.

This also allows something that initially seems pretty weird to be possible. Add the following line to your `compare` function:

```
val intComparable: Comparable<Int> = comparator
```

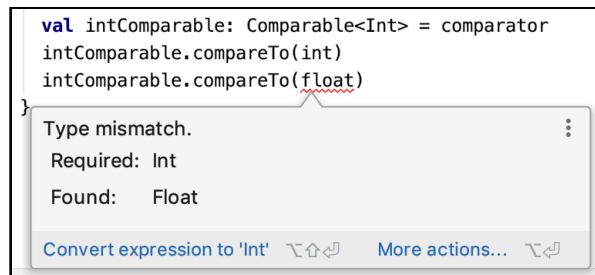
This compiles, which is somewhat counterintuitive — `Int` is a subtype of `Number`, not the other way around. But since the `Comparable<Number>` can definitely compare itself to `Int` values, it can also be used as a `Comparable<Int>`.

Being able to make this assignment means that, in exchange, `intComparable` will lose the ability to compare subtypes of `Number` other than `Int`.

Add the following lines to your `compare` function:

```
intComparable.compareTo(int)  
intComparable.compareTo(float)
```

You'll see that the first call works fine, but the second one rejects the type:



Type mismatch. Required: Int, Found: Float

Because you've made the generic type of Comparable more specific, you lose the ability to make comparisons to other subtypes.

Finally, now that you know how and why all these things work, it's useful to know the fancy-sounding technical names for all these different types of variance:

- **Covariant** types are the ones you've seen marked as `<out T>`. Because T can only be part of a return value, the relation of objects that take the same generic type is similar to that of supertypes and subtypes. You can assign something typed as `List<Int>` to a variable of type `List<Number>`, since `Int` is a subtype of `Number`.
- **Contravariant** types are the ones you've seen marked as `<in T>`. Since T can only be taken in as a parameter, you can assume the inverse relation to a subtype and supertype. You can assign something typed as `Comparable<Number>` to a variable of type `Comparable<Int>`, since `Number` is a supertype of `Int`.
- **Invariant** types are types that are simply marked as `<T>`. You cannot make inferences about relationships with other objects that take the same generic type, since they both take in and return objects of type T.

Challenges

1. Use generics to create a function that can print full names of a list of people as long as the objects representing them conform to a certain interface. The interface should allow you to access `String` values for `firstName` and `lastName`. Start with members of your family and your peers (or use fictional family members or peers if you'd prefer).

2. Create a `Vehicle` class that conforms to `Checkable` class and a `ShippingContainer` class that conforms to `Container<Vehicle>`, but that which only takes one vehicle at a time. Each vehicle should:
 - Know its own height in inches.
 - Know its model and brand names.
 - Display a combination of its model and brand names instead of its instance address when printed out using `println`.
 - Have a variable for a lambda, which allows callers to verify if the vehicle's height, in inches, is too big for a `Mover`'s truck by passing in the height of the vehicle to the lambda, then returning a `Boolean` value from the lambda of whether or not it will fit. (Hint: You can't do this as part of a constructor, since you won't have a reference to the `Mover` yet.)
 - Use that function as part of `checkIsOK()`.
3. Use the default constructor of `Mover<Vehicle>` and the functions you've already used to try to move three vehicles in `ShippingContainers`: A Yamaha Vino, which is 40 inches tall; a Toyota Corolla, which is 58 inches tall; and a Freightliner Cascadia, which is 150 inches tall. After doing that, answer the following questions:
 - How many of your vehicles does a mover — who is created using the default values — move?
 - Do you need to adjust the height of the moving vehicle in order to get all of the vehicles to be moved? If so, what is the height it needs to be adjusted to?
4. Create a `Mover` object that can move all the `Vehicle`, `BreakableThing` and `CheapThing` objects you've already created.
 - What is the type you need to pass to create a mover who can move all of these types of things?
 - Is there a kind of `Container` you can pass to this mover? If there is, what kind is it?

Key points

Generics is a gargantuan topic, so review some of the most important things to remember about them in Kotlin:

- **Generics** allow you to create classes or interfaces that operate on a type that is not known when your code for that class or interface is written.
- Generic programming can allow you to centralize pieces of functionality in a highly reusable and easily debuggable fashion.
- **Type erasure** means that, within a class or interface that takes a generic type, you won't have any information about that type at compile time unless you annotate the type with `reified` and `inline` the function.
- Allowing only in or out variance of a generic type allows you to restrict whether a generic type can be passed in to extensions or be returned from subclasses or other functions on a particular generic interface or class. This, in turn, allows both you and the compiler to make assumptions about how generic types relate to each other.

Where to go from here?

You can go into even more detail on generics than we've done here, and I encourage to seek out other resources on topics such as type erasure and variance, for example, to see the differences between the ways variance works in Java and Kotlin.

In the next chapter, Chapter 19, "Kotlin/Java Interoperability," we'll take a look at how Kotlin and Java work together, calling Kotlin code from Java and vice-versa.

Section IV: Intermediate Topics

You've made it to the final section of this book! In this section, you'll delve into some important but more intermediate topics to round out your Kotlin apprenticeship.